

LAPORAN PRAKTIKUM IF12D1
PEMROGRAMAN BERORIENTASI OBJEK
TUGAS PERTEMUAN 5

Disusun untuk Memenuhi Tugas Mata Kuliah Pemrograman Berorientasi Objek

Dosen Pengampu:

Mutaqin Akbar, S.Kom., M.T., MCE., MCF.



Disusun oleh :

Nama : DIMAS NURDIANSYAH
NIM : 251110058

UNIVERSITAS MERCU BUANA YOGYAKARTA
FAKULTAS TEKNOLOGI INFORMASI
PROGRAM STUDI INFORMATIKA
2025/2026

DAFTAR ISI

DAFTAR ISI	ii
DAFTAR GAMBAR	iii
A. Tugas	1
B. Alat dan Bahan	1
C. Jawaban.....	2
1. Rancangan Program	2
2. Source Code	3
2.1 Class Mahasiswa.java	3
2.2 Class MahasiswaReguler.java	4
2.3 Class MahasiswaBeasiswa.java	6
2.4 Class Main.java.....	7
3. Output.....	8
4. Alur Program	10
5. Kesimpulan.....	11

DAFTAR GAMBAR

Gambar 2.1. 1 Class Mahasiswa.....	3
Gambar 2.2. 1 Class MahasiswaReguler.....	4
Gambar 2.2. 2 Class MahasiswaReguler Lanjutan	4
Gambar 2.3. 1 Class MahasiswaBeasiswa.....	6
Gambar 2.4. 1 Class Main.....	7
Gambar 3. 1 Output Program.....	8

A. Tugas

buatlah sebuah system pencatatan nilai menggunakan konsep pewarisan bertingkat. ada 3 kelas yang saling berkaitan dalam satu hierarki.

- Kelas Mahasiswa adalah superclassnya. kelas ini menyimpan atribut umum yaitu NIM dan NAMA, dilengkapi getter setter dan metode tampilInfo() untuk mencetak data dasar mahasiswa.
- Kelas Mahasiswa Reguler mewarisi kelas mahasiswa menggunakan kata kunci extends. kelas ini menambahkan tiga atribut nilai akademik yaitu nilaiUTS, nilaiUAS dan nilaiASN. gunakan metode hitungNilaiAkhir() dengan rumus $30\% \text{ UTS} + 40\% \text{ UAS} + 30\% \text{ tugas}$. serta metode getGrade() yang mengembalikan/memberikan nilai akhir A sampai E. tampilInfo() juga harus di-override agar mencetak seluruh informasi nilai.
- Kelas MahasiswaBeasiswa mewarisi kelas MahasiswaReguler dan menambahkan atribut NilaiOrganisasi. Kalian harus override metode hitungNilaiAkhir() dengan rumus: nilai reguler dikali 90% ditambah nilaiOrganisasi dikali 10%. Yang menjadi tantangan di sini adalah mahasiswa WAJIB memanggil super.hitungNilaiAkhir() untuk mengambil hasil perhitungan dari kelas induk, sehingga tidak perlu menulis ulang rumus sebelumnya dari nol.
- Main Class bertugas membuat objek dari kedua subclass tersebut, kemudian panggil tampilInfo() dan menampilkan nilai akhir beserta grade dari masing masing objek.

B. Alat dan Bahan

1. Komputer
2. Modul
3. Java SDK (JDK)
4. Text Editor (VS Code)
5. Command Pemompt (CMD)

C. Jawaban

1. Rancangan Program

Pewarisan Bertingkat (Multi-level Inheritance) adalah konsep dalam Pemrograman Berorientasi Objek di mana sebuah kelas mewarisi kelas lain yang juga merupakan subclass dari kelas di atasnya, sehingga membentuk rantai hierarki bertingkat. Dalam Java, pewarisan diimplementasikan menggunakan keyword `extends`. Hierarki pewarisan pada program ini adalah:

Mahasiswa (superclass) => MahasiswaReguler (subclass 1) => MahasiswaBeasiswa (subclass 2)

Konsep penting lainnya yang digunakan dalam program ini adalah:

Override adalah kemampuan subclass untuk mendefinisikan ulang method yang sudah ada di superclass-nya. Dalam Java, override ditandai dengan anotasi `@Override`. Method yang di-override di subclass akan menggantikan method superclass ketika dipanggil melalui objek subclass. Keyword `super` digunakan untuk mengakses anggota (atribut atau method) dari superclass langsung. Pada program ini, `super.hitungNilaiAkhir()` digunakan di `MahasiswaBeasiswa` untuk mengambil hasil perhitungan dari `MahasiswaReguler` tanpa menulis ulang rumusnya, dan `super(nim, nama)` pada konstruktor digunakan untuk meneruskan nilai ke konstruktor superclass. Enkapsulasi diterapkan melalui penggunaan akses modifier `private` pada semua atribut, sehingga nilai atribut hanya dapat diakses dan diubah melalui method getter dan setter yang disediakan.

2. Source Code

2.1 Class Mahasiswa.java

```
Mahasiswa.java
1 public class Mahasiswa {
2     private String nim;
3     private String nama;
4
5     public Mahasiswa(String nim, String nama) {
6         this.nim = nim;
7         this.nama = nama;
8     }
9
10    public String getNim() {
11        return nim;
12    }
13
14    public void setNim(String nim) {
15        this.nim = nim;
16    }
17
18    public String getNama() {
19        return nama;
20    }
21
22    public void setNama(String nama) {
23        this.nama = nama;
24    }
25
26    public void tampilInfo() {
27        System.out.println("NIM      : " + nim);
28        System.out.println("Nama    : " + nama);
29    }
30 }
```

Gambar 2.1. 1 Class Mahasiswa

Penjelasan Kode Program :

Class Mahasiswa berfungsi sebagai superclass paling atas dalam hierarki pewarisan program ini. Kelas ini menjadi fondasi utama yang atribut dan method-nya akan diwarisi oleh seluruh kelas di bawahnya.

- `private String nim` dan `private String nama` => dua atribut utama untuk menyimpan data identitas mahasiswa. Menggunakan akses modifier `private` sebagai penerapan enkapsulasi, sehingga nilai kedua atribut ini hanya dapat diakses dan diubah melalui method getter dan setter yang disediakan.
- `public Mahasiswa(String nim, String nama)` => konstruktor yang digunakan untuk menginisialisasi nilai `nim` dan `nama` pada saat objek dibuat. Keyword `this` digunakan untuk membedakan antara atribut kelas dan parameter yang memiliki nama sama.
- `getNim()`, `setNim()`, `getNama()`, `setNama()` => empat method getter dan setter sebagai jembatan untuk mengakses dan mengubah nilai atribut `private` dari luar kelas.
- `public void tampilInfo()` => method untuk mencetak data dasar mahasiswa yaitu `NIM` dan `Nama` ke layar. Method ini akan di-override oleh subclass agar dapat mencetak data yang lebih lengkap sesuai jenis mahasiswanya.

2.2 Class MahasiswaReguler.java

```
MahasiswaReguler.java
1 public class MahasiswaReguler extends Mahasiswa {
2     private double nilaiUTS;
3     private double nilaiUAS;
4     private double nilaiASN;
5
6     public MahasiswaReguler(String nim, String nama, double nilaiUTS, double nilaiUAS, double nilaiASN) {
7         super(nim, nama);
8         this.nilaiUTS = nilaiUTS;
9         this.nilaiUAS = nilaiUAS;
10        this.nilaiASN = nilaiASN;
11    }
12
13    public double getNilaiUTS() {
14        return nilaiUTS;
15    }
16
17    public void setNilaiUTS(double nilaiUTS) {
18        this.nilaiUTS = nilaiUTS;
19    }
20
21    public double getNilaiUAS() {
22        return nilaiUAS;
23    }
24
25    public void setNilaiUAS(double nilaiUAS) {
26        this.nilaiUAS = nilaiUAS;
27    }
28
29    public double getNilaiASN() {
30        return nilaiASN;
31    }
32
33    public void setNilaiASN(double nilaiASN) {
34        this.nilaiASN = nilaiASN;
35    }
36 }
```

Gambar 2.2. 1 Class MahasiswaReguler

```
MahasiswaReguler.java
1 public class MahasiswaReguler extends Mahasiswa {
2
37    public double hitungNilaiAkhir() {
38        return (0.30 * nilaiUTS) + (0.40 * nilaiUAS) + (0.30 * nilaiASN);
39    }
40
41    public String getGrade() {
42        double nilaiAkhir = hitungNilaiAkhir();
43        if (nilaiAkhir >= 80.0) {
44            return "A";
45        } else if (nilaiAkhir >= 70.0) {
46            return "B";
47        } else if (nilaiAkhir >= 60.0) {
48            return "C";
49        } else if (nilaiAkhir >= 50.0) {
50            return "D";
51        } else {
52            return "E";
53        }
54    }
55
56    @Override
57    public void tampilInfo() {
58        super.tampilInfo();
59        System.out.println("Nilai UTS : " + nilaiUTS);
60        System.out.println("Nilai UAS : " + nilaiUAS);
61        System.out.println("Nilai Tugas : " + nilaiASN);
62        System.out.println("Nilai Akhir : " + String.format("%.2f", hitungNilaiAkhir()));
63        System.out.println("Grade : " + getGrade());
64    }
65 }
```

Gambar 2.2. 2 Class MahasiswaReguler Lanjutan

Penjelasan Kode Program :

Class MahasiswaReguler adalah subclass pertama yang mewarisi class Mahasiswa menggunakan keyword extends. Kelas ini menambahkan fungsionalitas berupa pencatatan dan perhitungan nilai akademik mahasiswa reguler.

- `public class MahasiswaReguler extends Mahasiswa` => mendeklarasikan bahwa kelas ini mewarisi seluruh atribut dan method dari kelas Mahasiswa. Dengan pewarisan ini, MahasiswaReguler secara otomatis memiliki atribut nim dan nama beserta getter setter-nya tanpa perlu menulis ulang.
- `private double nilaiUTS, nilaiUAS, nilaiASN` => tiga atribut baru untuk menyimpan nilai akademik. Menggunakan tipe double agar dapat menampung nilai desimal, dan private untuk enkapsulasi.
- `super(nim, nama)` => pemanggilan konstruktor superclass (Mahasiswa) untuk menginisialisasi atribut nim dan nama yang sudah dikelola di level atas. Ini adalah cara Java meneruskan nilai ke konstruktor kelas induk.
- `public double hitungNilaiAkhir()` => method untuk menghitung nilai akhir mahasiswa reguler dengan rumus $30\% \text{ UTS} + 40\% \text{ UAS} + 30\% \text{ Tugas}$. Bertipe double karena mengembalikan angka desimal hasil perhitungan.
- `public String getGrade()` => method yang memanggil `hitungNilaiAkhir()` lalu mengonversi hasilnya menjadi huruf mutu A hingga E menggunakan percabangan if-else. Bertipe String karena mengembalikan nilai berupa teks huruf.
- `@Override public void tampilInfo()` => override dari method `tampilInfo()` milik superclass. Pertama memanggil `super.tampilInfo()` untuk mencetak NIM dan Nama dari kelas induk, kemudian dilanjutkan dengan mencetak data nilai akademik yang baru ada di level kelas ini.

2.3 Class MahasiswaBeasiswa.java

```
1 public class MahasiswaBeasiswa extends MahasiswaReguler {
2     private double nilaiOrganisasi;
3
4     public MahasiswaBeasiswa(String nim, String nama, double nilaiUTS, double nilaiUAS, double nilaiASN, double nilaiOrganisasi) {
5         super(nim, nama, nilaiUTS, nilaiUAS, nilaiASN);
6         this.nilaiOrganisasi = nilaiOrganisasi;
7     }
8
9     public double getNilaiOrganisasi() {
10         return nilaiOrganisasi;
11     }
12
13     public void setNilaiOrganisasi(double nilaiOrganisasi) {
14         this.nilaiOrganisasi = nilaiOrganisasi;
15     }
16
17     @Override
18     public double hitungNilaiAkhir() {
19         double nilaiReguler = super.hitungNilaiAkhir();
20         return (nilaiReguler * 0.90) + (this.nilaiOrganisasi * 0.10);
21     }
22
23     @Override
24     public void tampilInfo() {
25         System.out.println("NIM          : " + getNim());
26         System.out.println("Nama          : " + getName());
27         System.out.println("Nilai UTS     : " + getNilaiUTS());
28         System.out.println("Nilai UAS     : " + getNilaiUAS());
29         System.out.println("Nilai Tugas   : " + getNilaiASN());
30         System.out.println("Nilai Organisasi : " + nilaiOrganisasi);
31         System.out.println("Nilai Akhir   : " + String.format("%.2f", hitungNilaiAkhir()));
32         System.out.println("Grade         : " + getGrade());
33     }
34 }
```

Gambar 2.3. 1 Class MahasiswaBeasiswa

Penjelasan Kode Program :

Class MahasiswaBeasiswa adalah subclass kedua sekaligus kelas paling bawah dalam hierarki. Kelas ini mewarisi MahasiswaReguler sehingga secara tidak langsung juga mewarisi Mahasiswa, menjadikan program ini menerapkan multi-level inheritance tiga tingkat.

- `public class MahasiswaBeasiswa extends MahasiswaReguler` => mendeklarasikan pewarisan dari MahasiswaReguler. Karena MahasiswaReguler sudah mewarisi Mahasiswa, maka MahasiswaBeasiswa secara otomatis memiliki semua atribut dan method dari kedua kelas di atasnya.
- `private double nilaiOrganisasi` => atribut tambahan khusus kelas ini untuk menyimpan nilai keaktifan organisasi mahasiswa penerima beasiswa.
- `super(nim, nama, nilaiUTS, nilaiUAS, nilaiASN)` => memanggil konstruktor MahasiswaReguler untuk meneruskan semua parameter ke kelas induk, sehingga semua atribut dari kelas atas terisi dengan benar.
- `@Override public double hitungNilaiAkhir()` => override dari method `hitungNilaiAkhir()` di MahasiswaReguler. Yang menjadi poin penting adalah penggunaan `super.hitungNilaiAkhir()` untuk mengambil hasil perhitungan nilai reguler dari kelas induk tanpa menulis ulang rumusnya, kemudian dikombinasikan dengan nilaiOrganisasi menggunakan rumus: $\text{nilai reguler} \times 90\% + \text{nilaiOrganisasi} \times 10\%$.

- `@Override public void tampilInfo()` => override `tampilInfo()` untuk menampilkan seluruh informasi mahasiswa beasiswa secara lengkap, termasuk `nilaiOrganisasi` yang disisipkan sebelum nilai akhir agar susunan output logis dan mudah dibaca.

2.4 Class Main.java

```

Main.java
1  public class Main {
2      public static void main(String[] args) {
3          System.out.println("Nama      : Dimas Nurdiansyah");
4          System.out.println("NIM       : 251110058\n");
5          System.out.println("        SISTEM PENCATATAN NILAI MAHASISWA        ");
6          System.out.println("=".repeat(45));
7          System.out.println();
8
9          //Mahasiswa Reguler
10         System.out.println("DATA MAHASISWA REGULER");
11         MahasiswaReguler mhsReguler = new MahasiswaReguler(
12             "251110017",
13             "AL Farel",
14             80.0, // UTS
15             82.5, // UAS
16             75.0 // ASN
17         );
18         mhsReguler.tampilInfo();
19         System.out.println();
20
21         // Mahasiswa Beasiswa
22         System.out.println("DATA MAHASISWA BEASISWA");
23         MahasiswaBeasiswa mhsBeasiswa = new MahasiswaBeasiswa(
24             "251110058",
25             "Dimas Nurdiansyah",
26             90.0, // UTS
27             91.0, // UAS
28             88.5, // ASN
29             95.0 // Nilai Organisasi
30         );
31         mhsBeasiswa.tampilInfo();
32     }
33 }

```

Gambar 2.4. 1 Class Main

Penjelasan Kode Program :

Class Main berfungsi sebagai entry point (titik masuk) program yang menjalankan seluruh alur eksekusi menggunakan method `main`.

- `public static void main(String[] args)` => method utama yang otomatis dijalankan pertama kali oleh Java Virtual Machine (JVM) saat program dieksekusi.
- `MahasiswaReguler mhsReguler = new MahasiswaReguler(...)` => membuat objek dari kelas `MahasiswaReguler` dan langsung menginisialisasi seluruh datanya melalui konstruktor dalam satu baris, mulai dari NIM, Nama, hingga ketiga nilai akademik.
- `MahasiswaBeasiswa mhsBeasiswa = new MahasiswaBeasiswa(...)` => membuat objek dari kelas `MahasiswaBeasiswa` dan menginisialisasi enam parameter

sekaligus termasuk nilaiOrganisasi yang menjadi pembeda dari mahasiswa reguler.

- mhsReguler.tampilInfo() dan mhsBeasiswa.tampilInfo() => memanggil method tampilInfo() pada masing-masing objek. Karena method ini di-override di setiap subclass, Java secara otomatis akan memanggil versi yang sesuai dengan tipe objeknya masing-masing — inilah penerapan konsep polymorphism dalam program ini.

3. Output

```
C:\Users\Bestdym\Data\Praktikum\OOP\Pertemuan5>javac *.java
C:\Users\Bestdym\Data\Praktikum\OOP\Pertemuan5>Java Main
Nama      : Dimas Nurdiansyah
NIM       : 251110058

=====
SISTEM PENCATATAN NILAI MAHASISWA
=====

DATA MAHASISWA REGULER
NIM      : 251110017
Nama     : AL Farel
Nilai UTS : 80.0
Nilai UAS : 82.5
Nilai Tugas : 75.0
Nilai Akhir : 79,50
Grade    : B

DATA MAHASISWA BEASISWA
NIM      : 251110058
Nama     : Dimas Nurdiansyah
Nilai UTS : 90.0
Nilai UAS : 91.0
Nilai Tugas : 88.5
Nilai Organisasi : 95.0
Nilai Akhir : 90,46
Grade    : A
```

Gambar 3. 1 Output Program

Penjelasan Cara Running Program :

Untuk menjalankan program, langkah pertama yang dilakukan adalah navigasi, yaitu masuk ke folder tempat kode disimpan melalui CMD menggunakan perintah cd path\ke\Pertemuan5. Perintah ini wajib dilakukan agar CMD mengetahui lokasi file program yang akan dikompilasi dan dijalankan. Setelah berada di dalam folder Pertemuan5, langkah selanjutnya adalah kompilasi, yaitu menerjemahkan kode Java menjadi file .class yang dapat dimengerti oleh komputer. Terdapat dua cara kompilasi yang dapat dilakukan:

- Cara pertama adalah kompilasi satu per satu, yaitu mengompilasi setiap file secara berurutan mulai dari Mahasiswa.java, kemudian MahasiswaReguler.java, lalu MahasiswaBeasiswa.java, dan terakhir Main.java menggunakan perintah javac di setiap filenya. Urutan ini sangat penting karena MahasiswaReguler bergantung pada Mahasiswa, dan MahasiswaBeasiswa bergantung pada MahasiswaReguler, sehingga jika urutannya terbalik maka compiler akan gagal menemukan kelas induknya.

- Kompilasi sekaligus menggunakan perintah `javac *.java`, di mana tanda `*.java` berarti mengompilasi semua file `.java` yang ada di dalam folder tersebut dalam satu perintah. Cara ini lebih cepat dan efisien karena tidak perlu mengetik nama file satu per satu, namun jika terjadi error maka semua pesan error dari berbagai file akan muncul bersamaan sehingga lebih sulit untuk dilacak.

Setelah proses kompilasi selesai tanpa error dan file `.class` terbentuk untuk setiap kelas, program dieksekusi menggunakan perintah `java Main`. Perintah ini menjalankan kelas `Main` yang di dalamnya terdapat method `main` sebagai titik masuk eksekusi program dan menghasilkan output berupa sistem pencatatan nilai mahasiswa yang menampilkan data dua objek mahasiswa secara terstruktur ke layar.

Penjelasan Output Program :

Output program menampilkan dua bagian data mahasiswa secara berurutan setelah header identitas dan judul sistem.

Bagian pertama adalah DATA MAHASISWA REGULER yang menampilkan data objek `mhsReguler` dari kelas `MahasiswaReguler`. NIM yang ditampilkan adalah 251110017 atas nama AL Farel. Nilai UTS sebesar 80.0, Nilai UAS sebesar 82.5, dan Nilai Tugas sebesar 75.0 dimasukkan melalui konstruktor saat objek dibuat. Nilai Akhir dihitung secara otomatis oleh method `hitungNilaiAkhir()` menggunakan rumus $30\% \text{ UTS} + 40\% \text{ UAS} + 30\% \text{ Tugas}$, yaitu $(0.30 \times 80.0) + (0.40 \times 82.5) + (0.30 \times 75.0) = 24.0 + 33.0 + 22.5 = 79.50$. Nilai akhir tersebut kemudian diproses oleh method `getGrade()` dan menghasilkan grade B karena berada pada rentang 70.0 hingga 79.99.

Bagian kedua adalah DATA MAHASISWA BEASISWA yang menampilkan data objek `mhsBeasiswa` dari kelas `MahasiswaBeasiswa`. NIM yang ditampilkan adalah 251110058 atas nama Dimas Nurdiansyah. Nilai UTS sebesar 90.0, Nilai UAS sebesar 91.0, Nilai Tugas sebesar 88.5, dan Nilai Organisasi sebesar 95.0. Nilai Akhir dihitung melalui method `hitungNilaiAkhir()` yang di-override di kelas `MahasiswaBeasiswa`. Perhitungannya dimulai dengan memanggil `super.hitungNilaiAkhir()` untuk mendapatkan nilai reguler terlebih dahulu, yaitu $(0.30 \times 90.0) + (0.40 \times 91.0) + (0.30 \times 88.5) = 27.0 + 36.4 + 26.55 = 89.95$. Hasil tersebut kemudian dikalkulasikan dengan rumus beasiswa yaitu $89.95 \times 0.90 + 95.0 \times 0.10 = 80.955 + 9.5 = 90.46$. Nilai akhir sebesar 90.46 kemudian diproses oleh method `getGrade()` dan menghasilkan grade A karena berada pada rentang 80.0 ke atas.

4. Alur Program

Program ini dirancang menggunakan konsep pewarisan bertingkat (*multi-level inheritance*) dengan tiga kelas yang tersusun secara hierarkis. Kelas paling atas adalah Mahasiswa yang berperan sebagai superclass utama. Kelas ini menyimpan dua atribut dasar yaitu nim dan nama bertipe String dengan akses modifier private sebagai penerapan enkapsulasi, dilengkapi dengan konstruktor, getter setter untuk masing-masing atribut, serta method tampilInfo() untuk mencetak data identitas mahasiswa ke layar.

Di tingkat kedua terdapat kelas MahasiswaReguler yang mewarisi kelas Mahasiswa menggunakan keyword extends. Kelas ini menambahkan tiga atribut nilai akademik yaitu nilaiUTS, nilaiUAS, dan nilaiASN bertipe double. Konstrukturnya memanggil super(nim, nama) untuk meneruskan inisialisasi atribut ke kelas induk. Kelas ini juga memiliki method hitungNilaiAkhir() dengan rumus $30\% \text{ UTS} + 40\% \text{ UAS} + 30\% \text{ Tugas}$, method getGrade() untuk mengonversi nilai akhir menjadi huruf mutu A hingga E, serta meng-override method tampilInfo() dengan memanggil super.tampilInfo() terlebih dahulu sebelum mencetak data nilai akademik tambahan.

Di tingkat paling bawah terdapat kelas MahasiswaBeasiswa yang mewarisi MahasiswaReguler, sehingga secara tidak langsung juga mewarisi seluruh atribut dan method dari Mahasiswa. Kelas ini menambahkan satu atribut baru yaitu nilaiOrganisasi bertipe double. Method hitungNilaiAkhir() di-override dengan rumus $\text{nilai reguler} \times 90\% + \text{nilaiOrganisasi} \times 10\%$, dan yang menjadi kewajiban di sini adalah pemanggilan super.hitungNilaiAkhir() untuk mengambil hasil perhitungan dari kelas induk tanpa menulis ulang rumus sebelumnya. Method tampilInfo() juga di-override agar seluruh informasi termasuk nilaiOrganisasi dapat ditampilkan secara lengkap dan terurut.

Seluruh program dijalankan melalui kelas Main yang berisi method main sebagai titik masuk eksekusi. Di dalamnya dibuat dua objek — satu dari MahasiswaReguler dan satu dari MahasiswaBeasiswa — yang masing-masing diinisialisasi melalui konstruktor, lalu dipanggil method tampilInfo()-nya. Java secara otomatis akan memanggil versi tampilInfo() yang sesuai dengan tipe objeknya masing-masing, yang merupakan penerapan konsep *polymorphism*. Alur eksekusi berjalan dari Main → pembuatan objek → pemanggilan konstruktor bertingkat via super → pemanggilan tampilInfo() → perhitungan nilai akhir via hitungNilaiAkhir() → penentuan grade via getGrade() → output tampil ke layar.

5. Kesimpulan

Program sistem pencatatan nilai mahasiswa ini berhasil diimplementasikan menggunakan konsep pewarisan bertingkat (*multi-level inheritance*) dalam bahasa pemrograman Java. Hierarki kelas tersusun dalam tiga tingkatan yaitu Mahasiswa sebagai superclass, MahasiswaReguler sebagai subclass tingkat pertama, dan MahasiswaBeasiswa sebagai subclass tingkat kedua yang mewarisi seluruh atribut dan method dari kedua kelas di atasnya.

Melalui program ini, beberapa konsep penting Pemrograman Berorientasi Objek berhasil diterapkan secara bersamaan. Konsep enkapsulasi diterapkan dengan menjadikan seluruh atribut bersifat private sehingga hanya dapat diakses melalui method getter dan setter. Konsep pewarisan diterapkan melalui keyword `extends` yang memungkinkan subclass mewarisi atribut dan method dari superclass tanpa perlu menulis ulang kode yang sama. Konsep override diterapkan pada method `tampilInfo()` dan `hitungNilaiAkhir()` agar setiap kelas dapat mencetak informasi dan menghitung nilai sesuai kebutuhannya masing-masing. Konsep polymorphism terlihat ketika Java secara otomatis memanggil versi `tampilInfo()` yang sesuai dengan tipe objeknya saat program dieksekusi. Selain itu, penggunaan keyword `super` terbukti sangat penting dalam program ini, baik untuk meneruskan nilai ke konstruktor kelas induk maupun untuk mengambil hasil perhitungan `hitungNilaiAkhir()` dari MahasiswaReguler di dalam MahasiswaBeasiswa tanpa perlu menulis ulang rumus dari awal.

Dengan demikian, program ini membuktikan bahwa penerapan konsep pewarisan bertingkat dapat membuat kode program menjadi lebih terstruktur, efisien, dan mudah dikembangkan karena setiap kelas memiliki tanggung jawabnya masing-masing sesuai dengan hierarkinya.